SLEEPING MACHINES • TUTORIAL 10

# Forward Feedback Trains Update Functions

Retained derivatives, parameter versions and causal aggregation

## Why this tutorial

A trainable learning rule can use forward observations to choose how the predictor should change. The next observation then teaches that rule. We derive what this local objective means, how to aggregate its feedback correctly, and which additional work is needed for a sparse asynchronous implementation.

## 1 Two sets of parameters

Let $\theta_t$ describe the predictor, $\Omega_t$ its optimizer state and $\psi_v$ version $v$ of an update function. After target $y_t$ arrives, form $g_t = \nabla_{\theta_t} \ell_t$ and issue

$$(\theta_{t+1}, \Omega_{t+1}) = U_{\psi_v}(\mathrm{sg}(\theta_t, \Omega_t, x_t, g_t)).$$

Here sg holds incoming values fixed when differentiating the update function. The prediction at the next event uses $\theta_{t+1}$ before seeing $y_{t+1}$. Retain the issued derivative $A_t = \partial\theta_{t+1}/\partial\psi_v$. Once that target arrives, its forward error supplies

$$a_t = A_t^{\mathsf{T}} g_{t+1}, \qquad g_{t+1} = \nabla_{\theta_{t+1}} \ell_{t+1}.$$

This is the chain rule for the declared local future-loss objective. It does not require the update function to imitate a conventional optimizer.

**Key idea**

The future target teaches the update which produced its predictor. The target may supervise learning after arrival, but cannot enter the earlier prediction or issued update.

## 2 A concrete geometric update

With positive semidefinite forward curvature $H_t$, let the function choose $\eta_t, \lambda_t, \beta_t$ and write

$$m' = \beta_t m + (1 - \beta_t) g_t, \quad r' = \beta_t r, \qquad \theta' = \theta - \eta_t (H_t + \lambda_t s_t I)^{-1} \frac{m'}{1 - r'},$$

where $s_t > 0$ is a detached curvature scale. A norm cap on the step is differentiated too. The full derivative includes the effect of damping on the solve and momentum on both $m'$ and $r'$. Its computational cost includes the solve, derivative action, retained tape and update-function optimizer.

## 3 Local teaching is not a full rollout gradient

The derivative above holds the incoming state fixed. Across several updates, the true rollout derivative also contains transported effects through predictor and optimizer state. For $S_t = (\theta_t, \Omega_t)$,

$$\frac{dS_{t+1}}{d\psi} = \frac{\partial U}{\partial \psi} + \frac{\partial U}{\partial S_t}\frac{dS_t}{d\psi}.$$

A local rule drops the second term. That creates a different objective, not a numerical error in the first term. Improving one next-target loss can still worsen cumulative loss if later optimizer-state effects are important.

## 4 Aggregate at a fixed function version

Pin $\psi_v$ while collecting $K$ feedback derivatives $a_1, \ldots, a_K$. Numeric learner state evolves; each issued local objective detaches its own incoming state. Commit their mean

$$\bar{a} = \frac{1}{K}\sum_i a_i, \qquad \mathrm{Cov}(\bar{a}) = \frac{1}{K^2}\sum_{i,j}\mathrm{Cov}(a_i, a_j).$$

This is the exact mean of the declared local derivatives. It is not a full $K$-target rollout derivative. Independent identical samples give a $1/K$ variance reduction; causal correlations can prevent it. Averaging does not remove truncation bias. Larger $K$ also delays regime adaptation.

## 5 Version and reset contracts

An issued tape belongs to the parameters which produced it. Consume its feedback before modifying those parameters, then issue the next tape at the new version. Release each consumed graph; only one need be retained in this streaming scheme. A document reset discards an unused pending graph and resets fast numerical state. Already observed accumulated derivatives can remain valid at their pinned version. Flush a partial batch before saving the final function checkpoint.

**Key idea**

Parameter versions and cancellation rules are part of the mathematical algorithm. A delayed derivative at an old version is not automatically the derivative of the current function.

## 6 What clockless sparse learning requires

An event packet needs its identity, addressed learner state, parameter version and derivative support. Feedback activates the derivative action and appropriate parameter blocks when it arrives. Economy requires a small support and cheap $A_t^{\mathsf{T}} g_{t+1}$ application: a short message carrying a dense global derivative does not make the computation sparse. Concurrent versions require immutable snapshots or an explicitly analysed stale-gradient rule.

## Exercises

**Exercise 1.** Why is feeding $g_{t+1}$ to the retained update derivative causal?

*Solution.* The next prediction has already been issued before its target is revealed. The gradient teaches the earlier update afterward; it cannot influence the scored prediction retroactively.

**Exercise 2.** If every derivative in a batch is the same random vector $a$, what is the covariance of their mean?

*Solution.* It is $\mathrm{Cov}(a)$, not $\mathrm{Cov}(a)/K$. All $K^2$ pairwise covariance terms are equal.

**Exercise 3.** Does exact local chain-rule teaching guarantee improvement of episode loss?

*Solution.* No. It optimizes a declared local surrogate, omits future state dependence, and uses finite noisy observations. Episode quality must be tested directly.

**Exercise 4.** A batch contains seven observed gradients when the episode ends. What should a mean-gradient protocol do?

*Solution.* Discard any unused pending tape, divide the accumulated gradients by seven, commit once and checkpoint. Dividing by the nominal batch size would change the stated objective.

**Where this appears in our work**

The first restricted native-language diagnostic uses a frozen two-layer temporal/keyed source, 64-dimensional lexical adaptation and a connected 25,027-parameter update function. Across four private 512-target episodes, online learning increases NLL by0.031139 relative to freezing the same pretrained function; its utility gate fails. This is a development result. A registered next stage averages sixteen fixed-version feedback derivatives. Neither diagnostic adds optional-route learning or establishes sparse hardware efficiency. See `CAUSAL_UPDATE_FUNCTION_FEEDBACK.md` and the source-bound results for full costs and scope.